

DATA SCIENCE CAPSTONE PROJECT

Employee Attrition Analytics & Retention Insights

Understanding the patterns behind employee turnover and identifying where retention efforts will have the most impact.

Prepared by Suvendu Mondal

Mindenious Edutech · Submission Deadline: 5 October 2026

Project Overview

Why this project matters

Objective

Analyze workforce records to understand the patterns associated with employee turnover and identify where retention efforts may improve.

Scope

Employee tenure, department, role, compensation band, performance, and workload data were combined into one working view, checked for quality, and used to derive attrition metrics, statistically test likely drivers, and flag current flight risk.

2,000

Employees (snapshot)

7

Departments

18

Roles

254

Recorded exits analyzed

Approach

A six-step pipeline from raw HR records to a decision-ready risk model

1 Prepare

Assemble employee records: tenure, department, role, compensation band, performance, and exit status.

2 Protect & Check

Remove duplicates, impute missing ratings, standardize labels — and keep only what's needed (no names, no emails).

3 Analyze

Break down attrition by department, tenure band, compensation band, and workload indicators.

4 Derive Metrics

Compute early-tenure attrition, trailing 12-month attrition rate, and workload/engagement gaps.

5 Test Relationships

Fit a logistic regression to identify which factors are significantly associated with exit, holding others constant.

6 Dashboard & Act

Score active employees for flight risk and summarize the strongest patterns into concrete recommendations.

Data Quality & Protection

Raw HR export was messy and identifier-heavy by design — here's what was fixed

50

Exact duplicate records removed

33

Missing performance ratings imputed (median)

7

Inconsistent department labels standardized

0

Names / emails / direct identifiers retained

Minimizing sensitive fields

Only employee_id is used as a key. No name, contact information, or other personally identifying fields are carried into the analysis — department, role, tenure, compensation band, performance, and workload indicators are sufficient to study attrition patterns.

Top-Line Attrition KPIs

Across the full workforce snapshot

12.70%

Overall Attrition Rate

4.05%

Trailing 12-Month Attrition

34.15%

Early-Tenure (<1yr) Attrition

6.2 hrs

Avg. Monthly Overtime

4.25 yrs

Avg. Tenure — Active

0.87 yrs

Avg. Tenure — Exited

6.48 / 10

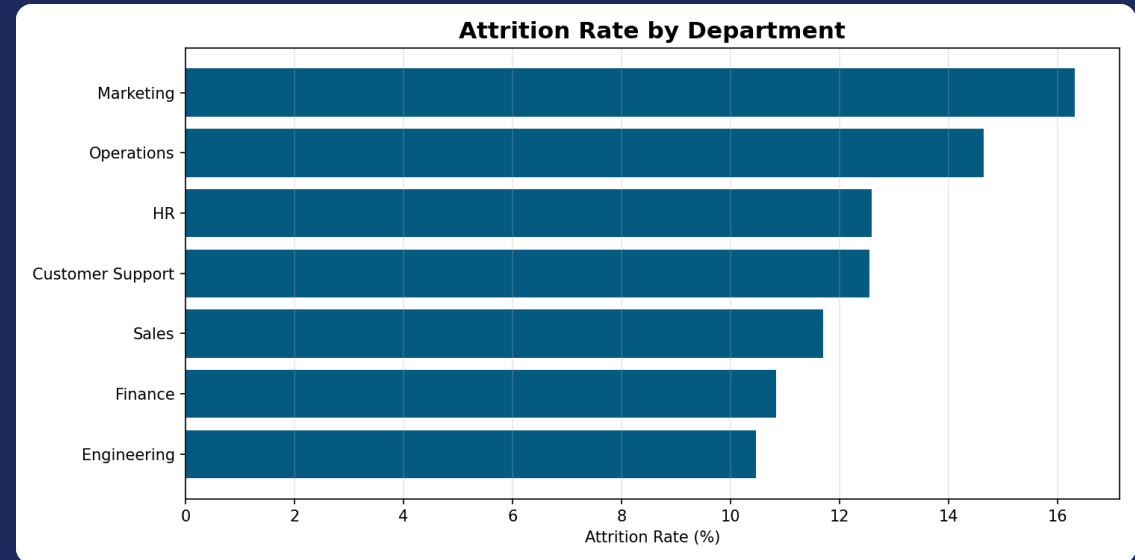
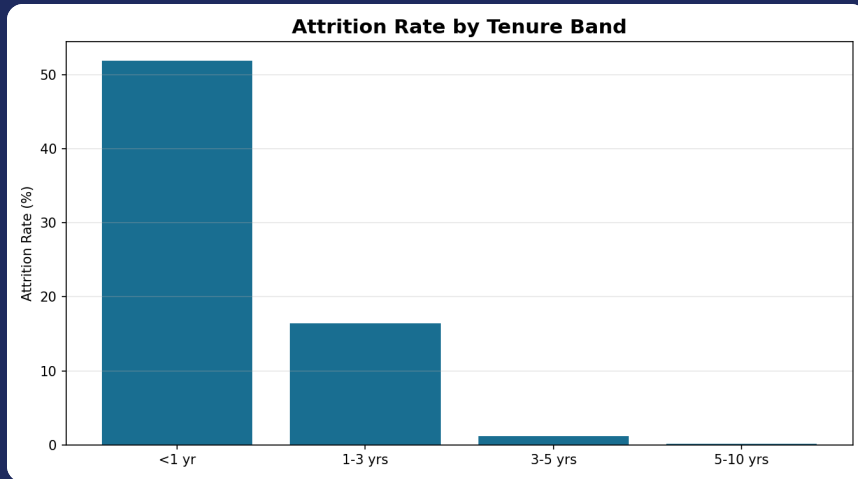
Avg. Engagement Score

2,000

Employees in Snapshot

Where Attrition Concentrates

Tenure is the dominant signal; department differences are smaller but real

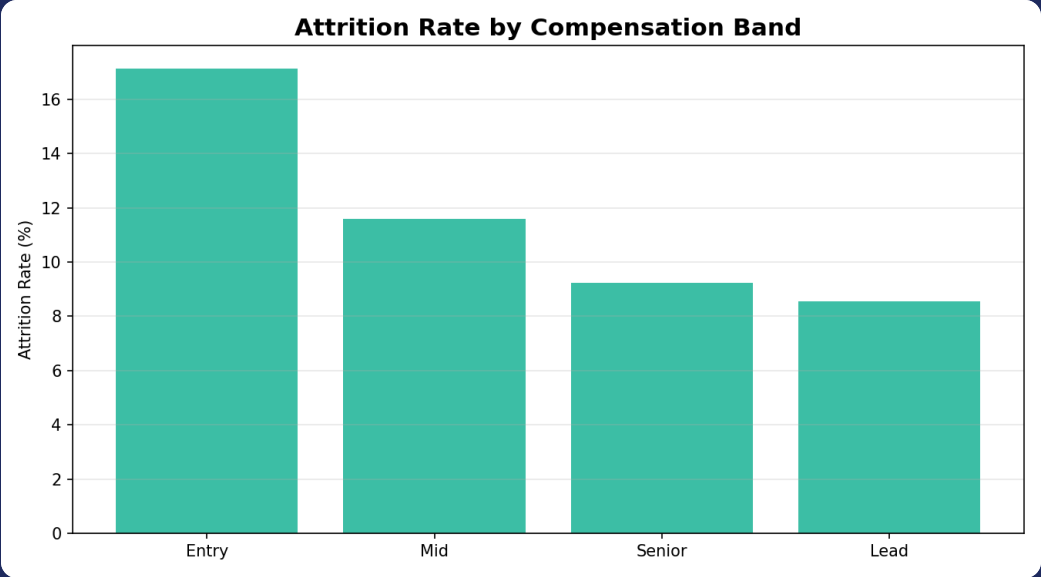


Takeaway:

Over half of employees with under a year of tenure ultimately exit, versus near-zero once past the 5-year mark. Marketing (16.3%) and Operations (14.7%) run hottest by department, while Engineering (10.5%) and Finance (10.8%) run coolest.

Compensation & Performance Patterns

Attrition falls steadily as pay band rises



Comp. Band	Headcount	Avg Salary	Attrition
Entry	683	\$35,153	17.13%
Mid	707	\$53,805	11.60%
Senior	423	\$79,790	9.22%
Lead	187	\$119,047	8.56%

Overtime & engagement gap

Exited employees averaged 7.1 monthly overtime hours vs. 6.0 for those still active, and scored lower on engagement (6.07 vs. 6.54 / 10).

What Actually Drives Attrition?

Logistic regression — effect of each factor holding the others constant

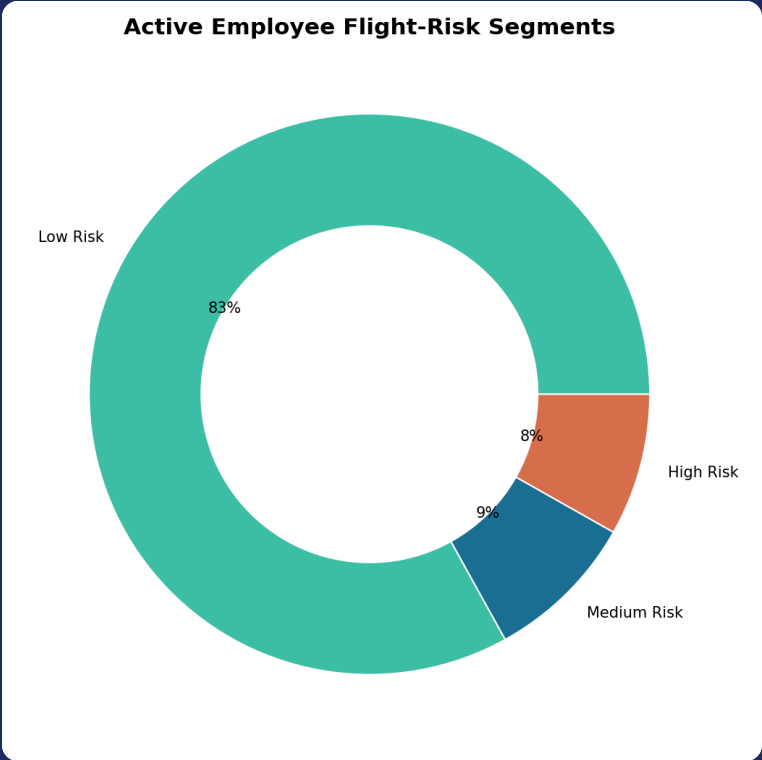
Factor	Odds Ratio	Interpretation	Significance
Tenure (per year)	0.268	Each additional year cuts exit odds by ~73%	p < 0.001
Entry band (vs. Lead)	2.817	Entry-band employees are ~2.8x more likely to exit	p = 0.003
Mid band (vs. Lead)	1.906	Directionally higher, not conclusive	p = 0.068
Performance rating	0.629	Each rating point lowers exit odds by ~37%	p < 0.001
Monthly overtime (per hr)	1.057	Each extra hour raises exit odds ~5.7%	p = 0.003
Engagement score	0.843	Each point lowers exit odds by ~16%	p = 0.001

Model fit:

Pseudo $R^2 = 0.44$ on 2,000 employees — tenure, compensation band, performance, overtime, and engagement together explain a substantial share of who exits.

Flight-Risk Segmentation

Model-scored risk across 1,746 currently active employees



Segment	Headcount	Meaning
Low Risk	1,450 (83%)	Stable — no action needed
Medium Risk	153 (9%)	Watch list — check in during 1:1s
High Risk	143 (8%)	Prioritize for stay interviews

High-risk concentration

Finance (32) and Customer Support (24) hold the most high-risk employees in absolute terms — worth a targeted retention push in those two teams first.

Recommendations

Data-backed next steps

- 1 Fix the first-year experience**
Early-tenure (<1yr) attrition sits at 34%, over half of all <1yr employees eventually leave — strengthen onboarding, mentorship, and 30/60/90-day check-ins.
- 2 Re-examine entry-band pay & growth path**
Entry-band employees are 2.8x more likely to exit than Lead-band, even after controlling for tenure and performance — review starting pay and promotion timelines.
- 3 Run stay interviews for the 143 high-risk actives**
Concentrated in Finance and Customer Support — a proactive conversation is cheaper than a replacement hire.
- 4 Cap chronic overtime**
Each extra monthly overtime hour raises exit odds ~5.7% — audit workload in teams running consistently high overtime.
- 5 Protect engagement, not just pay**
Engagement score is independently protective (OR 0.84) — recognition and manager check-ins matter even where compensation can't move.

Tech Stack

✓ **Python (pandas, numpy, statsmodels)**

Data prep, cleaning, and logistic regression

✓ **scikit-learn / statsmodels**

Attrition risk scoring for active employees

✓ **Matplotlib**

Dashboard-ready charts

Thank You

Suvendu Mondal · Data Science Capstone · Mindenious Edutech

APPENDIX

Full Code & Execution Output

Every code cell from the analysis notebook, run end-to-end, with its printed results and charts shown inline below.

Employee Attrition Analytics & Retention Insights

Data Science Capstone Project — Mindenious Edutech Prepared by Suvendu Mondal

This notebook walks through the full analysis pipeline used for the capstone brief:

1. Generate/assemble the employee records (tenure, department, role, compensation band, performance, exit status)
2. Check data quality and protect sensitive fields (drop duplicates, impute missing ratings, standardize labels, minimize identifiers)
3. Build top-line and derived attrition metrics (overall, trailing 12-month, early-tenure)
4. Analyze attrition by department, tenure band, compensation band, and performance
5. Fit a logistic regression to test which factors are significantly associated with attrition
6. Score active employees for flight risk and summarize patterns into recommendations

Runs standalone — no file uploads needed. A synthetic but realistic dataset is generated in Step 1 so the notebook is fully reproducible. No employee names, emails, or other direct identifiers are generated or used anywhere in the pipeline.

Step 1 — Generate the raw dataset

Simulates an HRIS export (with intentional dirty data: duplicates, missing performance ratings, inconsistent department labels) so the cleaning step is genuinely exercised. Exit risk is generated from a synthetic function of tenure, compensation band, performance, overtime, and engagement, so the later regression has real signal to recover.

In [1]:

```
# Install/import dependencies (Colab already has most of these)
# !pip install -q faker statsmodels --break-system-packages > /dev/null 2>&1

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
from faker import Faker
import random

random.seed(42)
np.random.seed(42)
fake = Faker()
Faker.seed(42)

plt.rcParams["figure.facecolor"] = "white"
plt.rcParams["axes.spines.top"] = False
plt.rcParams["axes.spines.right"] = False

# Step 1 -- Generate the raw dataset
# Simulates an HRIS export (with intentional dirty data: duplicates, missing
# values, inconsistent labels) so the cleaning step is genuinely exercised.

N_EMPLOYEES = 2000
SNAPSHOT_DATE = pd.Timestamp("2026-09-01")

DEPARTMENTS = {
    "Engineering": ["Software Engineer", "QA Engineer", "DevOps Engineer", "Engineering Manager"],
    "Sales": ["Sales Executive", "Account Manager", "Sales Manager"],
    "Customer Support": ["Support Associate", "Support Team Lead"],
    "Marketing": ["Marketing Associate", "Content Strategist", "Marketing Manager"],
    "Finance": ["Financial Analyst", "Accountant", "Finance Manager"],
    "HR": ["HR Associate", "Recruiter", "HR Manager"],
    "Operations": ["Operations Analyst", "Operations Manager"],
}

COMP_BANDS = ["Entry", "Mid", "Senior", "Lead"]
COMP_BAND_RANGE = {"Entry": (28000, 42000), "Mid": (42000, 65000),
                    "Senior": (65000, 95000), "Lead": (95000, 140000)}

rows = []
for i in range(N_EMPLOYEES):
    dept = random.choice(list(DEPARTMENTS.keys()))
    role = random.choice(DEPARTMENTS[dept])
    comp_band = random.choices(COMP_BANDS, weights=[0.35, 0.35, 0.2, 0.1])[0]
```

```

lo, hi = COMP_BAND_RANGE[comp_band]
salary = round(np.random.uniform(lo, hi), 2)

# Hire date spread over the last ~8 years; tenure-at-snapshot drives risk
hire_date = SNAPSHOT_DATE - pd.Timedelta(days=int(np.random.uniform(30, 8 * 365.25)))
tenure_at_snapshot = (SNAPSHOT_DATE - hire_date).days / 365.25

performance = np.clip(np.round(np.random.normal(3.2, 0.9)), 1, 5)
overtime_hours = max(0, np.random.normal(6, 5))
projects_count = max(1, int(np.random.poisson(3)))
engagement_score = np.clip(np.round(np.random.normal(6.5, 1.8), 1), 1, 10)

# Synthetic attrition logic: higher risk for short tenure, low pay band,
# low performance, high overtime, low engagement.
risk = (
    -0.35 * tenure_at_snapshot
    + (1.1 if comp_band == "Entry" else 0.4 if comp_band == "Mid" else 0.0)
    + (0.8 if performance <= 2 else 0.0)
    + 0.05 * overtime_hours
    - 0.18 * engagement_score
    + np.random.normal(0, 1.1)
)
exit_prob = 1 / (1 + np.exp(-(risk - 1.0)))
exited = np.random.random() < exit_prob

exit_date = None
tenure_years = tenure_at_snapshot
if exited:
    # Exit date falls somewhere between hire and snapshot, skewed toward
    # earlier tenure for lower engagement/comp employees (early quits).
    frac = np.clip(np.random.beta(1.5, 3.0), 0.02, 1.0)
    exit_date = hire_date + pd.Timedelta(days=int(frac * (SNAPSHOT_DATE - hire_date).days))
    tenure_years = max(0.02, (exit_date - hire_date).days / 365.25)

rows.append({
    "employee_id": f"EMP{i+1:05d}",
    "department": random.choice([dept, dept.upper(), f" {dept}"]), # dirty on purpose
    "role": role,
    "hire_date": hire_date.date(),
    "tenure_years": tenure_years,
    "compensation_band": comp_band,
    "annual_salary": salary,
    "performance_rating": int(performance) if random.random() > 0.015 else None,
    "overtime_hours_monthly": round(overtime_hours, 1),
    "projects_count": projects_count,
    "engagement_score": engagement_score,
    "exit_status": "Exited" if exited else "Active",
    "exit_date": exit_date.date() if exit_date is not None else None,
})

emp_df = pd.DataFrame(rows)

# Inject ~2.5% exact duplicate rows (dirty data)
dupe_idx = emp_df.sample(frac=0.025, random_state=42).index
emp_df = pd.concat([emp_df, emp_df.loc[dupe_idx]], ignore_index=True)

print(f"Raw records: {len(emp_df):,}")
print(emp_df.head())
emp_df.to_pickle("/home/claude/attrition/raw.pkl")

```

Raw records: 2,050

	employee_id	department	...	exit_status	exit_date
0	EMP00001	HR	...	Exited	2020-12-26
1	EMP00002	Sales	...	Active	None
2	EMP00003	Engineering	...	Exited	2022-11-09
3	EMP00004	Finance	...	Active	None
4	EMP00005	HR	...	Active	None

[5 rows x 13 columns]

Step 2 — Data quality checks & cleaning

Drop exact duplicates, impute missing performance ratings with the median, standardize department text, and derive tenure bands and exit flags. Only employee_id is retained as a key — no name, email or contact field is ever created.

In [2]:

```
import pandas as pd
import numpy as np

df = pd.read_pickle("/home/claude/attrition/raw.pkl")

# Step 2 -- Data quality checks & cleaning
# No names, emails, or other direct identifiers are collected -- employee_id
# is the only key, keeping the dataset minimal per data-protection guidance.

before = len(df)
df = df.drop_duplicates()
print(f"Dropped {before - len(df)} exact duplicate rows")

df["department"] = df["department"].str.strip().str.title()
df["department"] = df["department"].replace({"Hr": "HR"})

missing_perf = df["performance_rating"].isna().sum()
median_perf = df["performance_rating"].median()
df["performance_rating"] = df["performance_rating"].fillna(median_perf)
print(f"Imputed {missing_perf} missing performance ratings with median ({median_perf:.0f})")

df["hire_date"] = pd.to_datetime(df["hire_date"])
df["exit_date"] = pd.to_datetime(df["exit_date"])

TENURE_BINS = [0, 1, 3, 5, 10, 100]
TENURE_LABELS = ["<1 yr", "1-3 yrs", "3-5 yrs", "5-10 yrs", "10+ yrs"]
df["tenure_band"] = pd.cut(df["tenure_years"], bins=TENURE_BINS, labels=TENURE_LABELS, right=False)

df["is_exited"] = (df["exit_status"] == "Exited").astype(int)
df["early_tenure_exit"] = ((df["is_exited"] == 1) & (df["tenure_years"] < 1)).astype(int)
df["exit_month"] = df["exit_date"].values.astype("datetime64[M]")

print(f"\nClean dataset: {len(df):,} rows, {df['employee_id'].nunique():,} unique employees")
print(df[["department", "role", "tenure_band", "compensation_band",
          "performance_rating", "exit_status"]].head())

df.to_pickle("/home/claude/attrition/clean.pkl")
```

Dropped 50 exact duplicate rows

Imputed 33 missing performance ratings with median (3)

Clean dataset: 2,000 rows, 2,000 unique employees

	department	role	... performance_rating	exit_status
0	HR	HR Associate	...	4.0 Exited
1	Sales	Sales Manager	...	4.0 Active
2	Engineering	Engineering Manager	...	3.0 Exited
3	Finance	Finance Manager	...	4.0 Active
4	HR	HR Manager	...	2.0 Active

[5 rows x 6 columns]

Step 3 — Top-line KPIs & derived metrics

Overall attrition, trailing 12-month attrition, and early-tenure (<1yr) attrition are computed separately, since they answer different retention questions.

In [3]:

```
import pandas as pd

df = pd.read_pickle("/home/claude/attrition/clean.pkl")

# Step 3 -- Top-line KPIs & derived attrition metrics

total_employees = len(df)
total_exits = df["is_exited"].sum()
overall_attrition = total_exits / total_employees

early_exits = df["early_tenure_exit"].sum()
early_tenure_pop = (df["tenure_years"] < 1).sum() + early_exits # exited-before-1yr + still-under-1yr actives
early_tenure_rate = early_exits / max(1, (df["tenure_years"] < 1).sum() + early_exits)

avg_tenure_active = df.loc[df["exit_status"] == "Active", "tenure_years"].mean()
avg_tenure_exited = df.loc[df["exit_status"] == "Exited", "tenure_years"].mean()
avg_engagement = df["engagement_score"].mean()
avg_overtime = df["overtime_hours_monthly"].mean()

# Annualized attrition rate: exits in the trailing 12 months / avg headcount
snapshot = pd.Timestamp("2026-09-01")
trailing_12mo_exits = df[(df["exit_date"] >= snapshot - pd.Timedelta(days=365)) & (df["is_exited"] == 1)].shape[0]
annual_attrition_rate = trailing_12mo_exits / total_employees

kpis = {
    "Total Employees (snapshot)": f"{total_employees:,}",
    "Total Exits (all time)": f"{total_exits:,}",
    "Overall Attrition Rate": f"{overall_attrition*100:.2f}%",
    "Trailing 12-Mo Attrition Rate": f"{annual_attrition_rate*100:.2f}%",
    "Early-Tenure (<1yr) Attrition Rate": f"{early_tenure_rate*100:.2f}%",
    "Avg Tenure (Active)": f"{avg_tenure_active:.2f} yrs",
    "Avg Tenure (Exited)": f"{avg_tenure_exited:.2f} yrs",
    "Avg Engagement Score": f"{avg_engagement:.2f} / 10",
    "Avg Monthly Overtime": f"{avg_overtime:.1f} hrs",
}

for k, v in kpis.items():
    print(f"{k:<36}: {v}")

import json
with open("/home/claude/attrition/kpis.json", "w") as f:
    json.dump(kpis, f)
```

Total Employees (snapshot)	: 2,000
Total Exits (all time)	: 254
Overall Attrition Rate	: 12.70%
Trailing 12-Mo Attrition Rate	: 4.05%
Early-Tenure (<1yr) Attrition Rate	: 34.15%
Avg Tenure (Active)	: 4.25 yrs
Avg Tenure (Exited)	: 0.87 yrs
Avg Engagement Score	: 6.48 / 10
Avg Monthly Overtime	: 6.2 hrs

Step 4 — Attrition by department, tenure, compensation, performance

Breaks the workforce down along the four dimensions the brief calls out, plus monthly exit volume and a workload/engagement comparison between active and exited employees.

In [4]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker

df = pd.read_pickle("/home/claude/attrition/clean.pkl")
CH = "/home/claude/attrition/charts"

# Step 4 -- Attrition by department, tenure band, compensation band, performance

dept_perf = (df.groupby("department")
              .agg(headcount=("employee_id", "count"), exits=("is_exited", "sum"))
              .reset_index())
dept_perf["attrition_rate"] = dept_perf["exits"] / dept_perf["headcount"] * 100
dept_perf = dept_perf.sort_values("attrition_rate", ascending=False)

fig, ax = plt.subplots(figsize=(10, 5))
ax.barh(dept_perf["department"], dept_perf["attrition_rate"], color="#065A82")
ax.set_title("Attrition Rate by Department", fontsize=15, fontweight="bold")
ax.set_xlabel("Attrition Rate (%)")
ax.invert_yaxis()
ax.grid(axis="x", alpha=0.3)
plt.tight_layout()
plt.savefig(f"{CH}/attrition_by_department.png", dpi=150)
plt.close()
print(dept_perf)

tenure_perf = (df.groupby("tenure_band", observed=True)
               .agg(headcount=("employee_id", "count"), exits=("is_exited", "sum"))
               .reset_index())
tenure_perf["attrition_rate"] = tenure_perf["exits"] / tenure_perf["headcount"] * 100

fig, ax = plt.subplots(figsize=(9, 5))
ax.bar(tenure_perf["tenure_band"], tenure_perf["attrition_rate"], color="#1C7293")
ax.set_title("Attrition Rate by Tenure Band", fontsize=15, fontweight="bold")
ax.set_ylabel("Attrition Rate (%)")
ax.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.savefig(f"{CH}/attrition_by_tenure.png", dpi=150)
plt.close()
print(tenure_perf)

comp_perf = (df.groupby("compensation_band", observed=True)
              .agg(headcount=("employee_id", "count"), exits=("is_exited", "sum"),
                   avg_salary=("annual_salary", "mean"))
              .reindex(["Entry", "Mid", "Senior", "Lead"]).reset_index())
comp_perf["attrition_rate"] = comp_perf["exits"] / comp_perf["headcount"] * 100

fig, ax = plt.subplots(figsize=(9, 5))
ax.bar(comp_perf["compensation_band"], comp_perf["attrition_rate"], color="#3FBFA6")
ax.set_title("Attrition Rate by Compensation Band", fontsize=15, fontweight="bold")
ax.set_ylabel("Attrition Rate (%)")
ax.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.savefig(f"{CH}/attrition_by_comp_band.png", dpi=150)
plt.close()
print(comp_perf)

perf_perf = (df.groupby("performance_rating")
              .agg(headcount=("employee_id", "count"), exits=("is_exited", "sum"))
              .reset_index())
perf_perf["attrition_rate"] = perf_perf["exits"] / perf_perf["headcount"] * 100
```

```

monthly_exits = (df.dropna(subset=["exit_month"])
                 .groupby("exit_month").size().reset_index(name="exits")
                 .sort_values("exit_month"))

fig, ax = plt.subplots(figsize=(11, 5))
ax.plot(monthly_exits["exit_month"], monthly_exits["exits"], marker="o", color="#D9704F", linewidth=2)
ax.set_title("Monthly Exit Volume", fontsize=15, fontweight="bold")
ax.set_ylabel("Exits")
ax.grid(axis="y", alpha=0.3)
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig(f"{CH}/monthly_exits.png", dpi=150)
plt.close()
print(perf_perf)

# Workload indicator: overtime hours, active vs exited
overtime_cmp = df.groupby("exit_status")["overtime_hours_monthly"].mean()
engagement_cmp = df.groupby("exit_status")["engagement_score"].mean()
print("\nAvg monthly overtime by status:\n", overtime_cmp)
print("\nAvg engagement score by status:\n", engagement_cmp)

dept_perf.to_pickle(f"{CH}/../dept_perf.pkl")
tenure_perf.to_pickle(f"{CH}/../tenure_perf.pkl")
comp_perf.to_pickle(f"{CH}/../comp_perf.pkl")

```

	department	headcount	exits	attrition_rate	
4	Marketing	288	47	16.319444	
5	Operations	273	40	14.652015	
3	HR	278	35	12.589928	
0	Customer Support	279	35	12.544803	
6	Sales	282	33	11.702128	
2	Finance	323	35	10.835913	
1	Engineering	277	29	10.469314	
	tenure_band	headcount	exits	attrition_rate	
0	<1 yr	324	168	51.851852	
1	1-3 yrs	481	79	16.424116	
2	3-5 yrs	487	6	1.232033	
3	5-10 yrs	708	1	0.141243	
	compensation_band	headcount	exits	avg_salary	attrition_rate
0	Entry	683	117	35153.234436	17.130307
1	Mid	707	82	53804.815898	11.598303
2	Senior	423	39	79789.657565	9.219858
3	Lead	187	16	119046.592995	8.556150
	performance_rating	headcount	exits	attrition_rate	
0	1.0	58	8	13.793103	
1	2.0	384	77	20.052083	
2	3.0	831	103	12.394705	
3	4.0	577	57	9.878683	
4	5.0	150	9	6.000000	

```

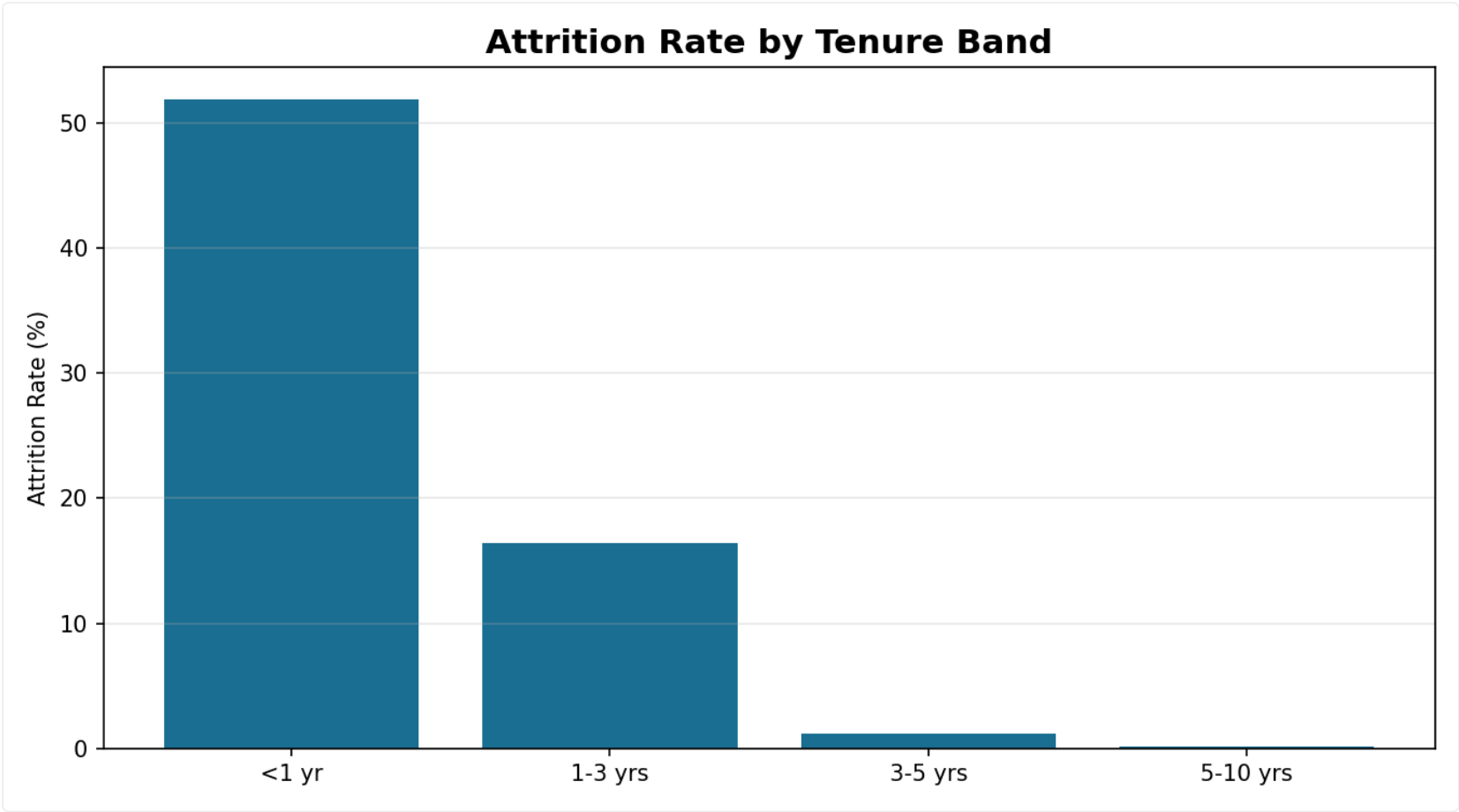
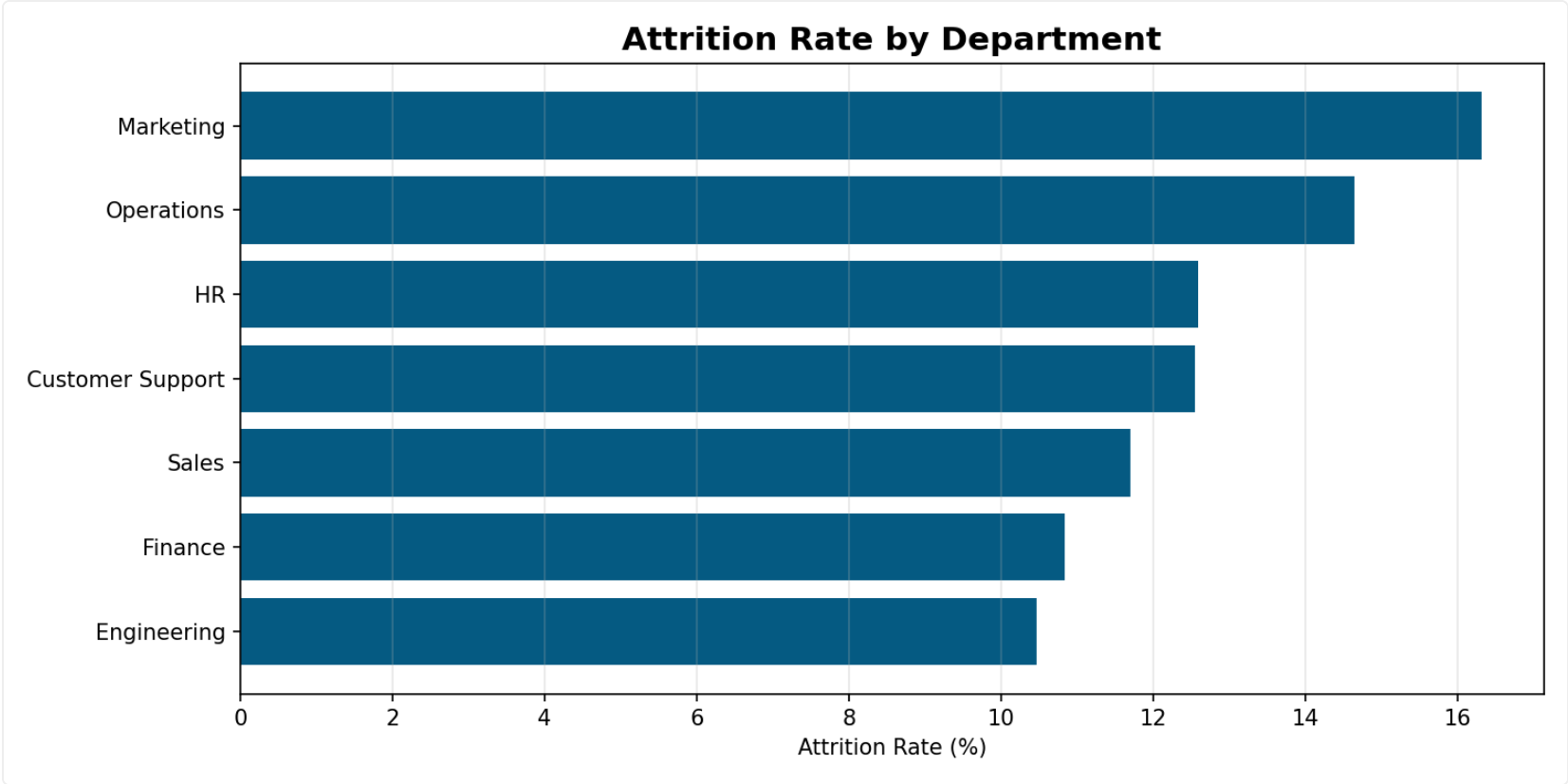
Avg monthly overtime by status:
exit_status
Active      6.041123
Exited      7.110236
Name: overtime_hours_monthly, dtype: float64

```

```

Avg engagement score by status:
exit_status
Active      6.538603
Exited      6.071654
Name: engagement_score, dtype: float64

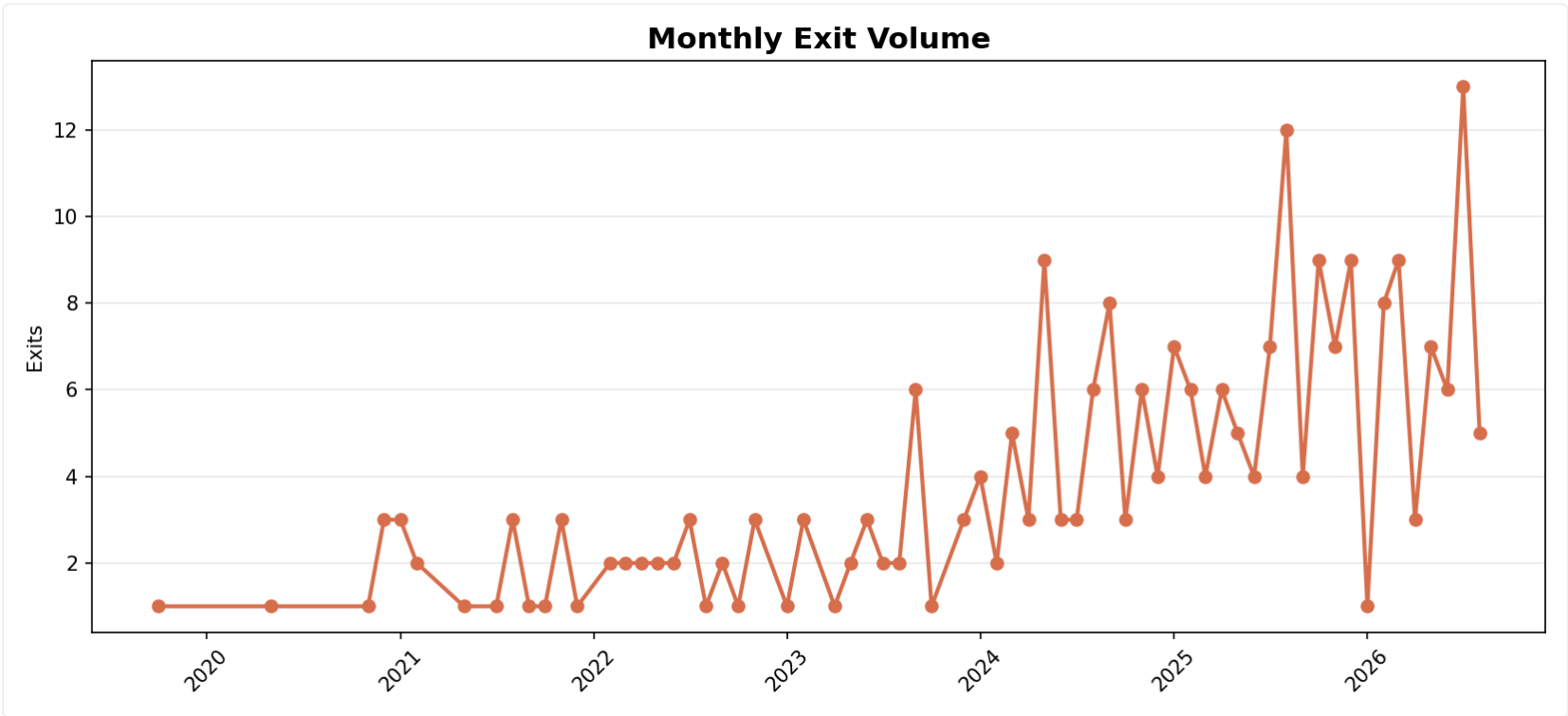
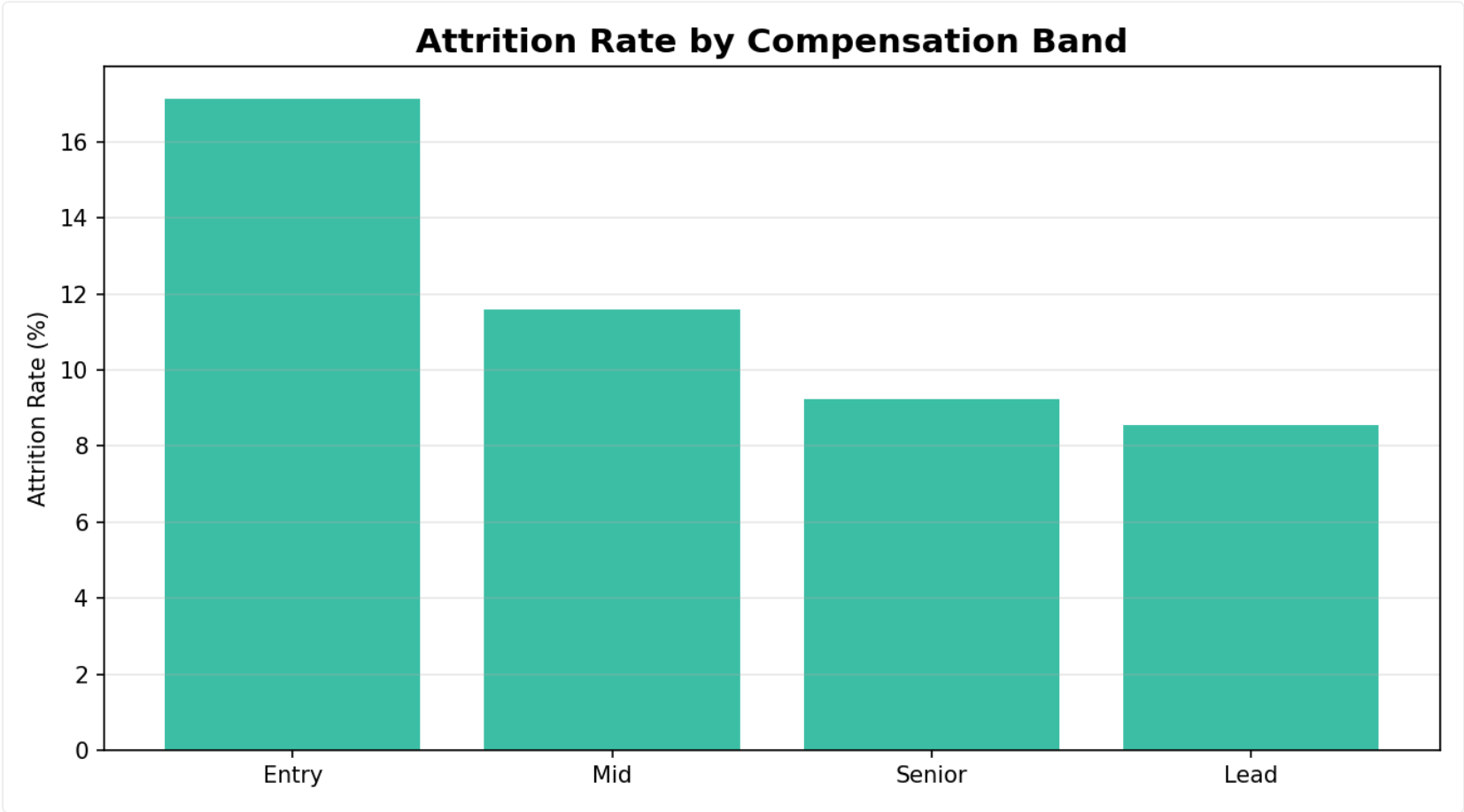
```



(continued)

In [4 (cont.)]:

```
# ... continued from previous cell: compensation-band and performance charts
```



Step 5 — Logistic regression: what drives attrition?

A logistic regression of exit status on tenure, compensation band, performance rating, monthly overtime, and engagement score, so each factor's effect can be read holding the others constant.

In [5]:

```
import pandas as pd
import numpy as np
import statsmodels.api as sm
import statsmodels.formula.api as smf

df = pd.read_pickle("/home/claude/attrition/clean.pkl")

# Step 5 -- Logistic regression: which factors are significantly associated
# with attrition, holding the others constant?

model_df = df.copy()
model_df["comp_band_code"] = pd.Categorical(
    model_df["compensation_band"], categories=["Entry", "Mid", "Senior", "Lead"], ordered=True
)

formula = ("is_exited ~ tenure_years + C(comp_band_code, Treatment(reference='Lead')) "
          "+ performance_rating + overtime_hours_monthly + engagement_score")
logit_model = smf.logit(formula, data=model_df).fit(dis=0)
print(logit_model.summary())

# Odds ratios for interpretability
odds_ratios = np.exp(logit_model.params)
conf = np.exp(logit_model.conf_int())
conf.columns = ["OR_2.5%", "OR_97.5%"]
result_table = pd.concat([odds_ratios.rename("Odds Ratio"), conf,
                          logit_model.pvalues.rename("p-value")], axis=1)
print("\nOdds ratios:\n", result_table.round(3))

result_table.to_pickle("/home/claude/attrition/odds_ratios.pkl")

with open("/home/claude/attrition/logit_summary.txt", "w") as f:
    f.write(str(logit_model.summary()))
```

Logit Regression Results						
=====						
Dep. Variable:	is_exited	No. Observations:	2000			
Model:	Logit	Df Residuals:	1992			
Method:	MLE	Df Model:	7			
Date:	Sun, 27 Sep 2026	Pseudo R-squ.:	0.4422			
Time:	09:07:48	Log-Likelihood:	-424.66			
converged:	True	LL-Null:	-761.29			
Covariance Type:	nonrobust	LLR p-value:	4.034e-141			
=====						
		coef	std err	z	P> z	[0.025 0.975]

Intercept		2.1986	0.556	3.953	0.000	1.109 3.289
C(comp_band_code, Treatment(reference='Lead'))[T.Entry]		1.0356	0.347	2.987	0.003	0.356 1.715
C(comp_band_code, Treatment(reference='Lead'))[T.Mid]		0.6449	0.354	1.823	0.068	-0.048 1.338
C(comp_band_code, Treatment(reference='Lead'))[T.Senior]		0.1506	0.378	0.398	0.690	-0.590 0.892
tenure_years		-1.3177	0.091	-14.404	0.000	-1.497 -1.138
performance_rating		-0.4638	0.097	-4.761	0.000	-0.655 -0.273
overtime_hours_monthly		0.0551	0.019	2.941	0.003	0.018 0.092
engagement_score		-0.1702	0.052	-3.284	0.001	-0.272 -0.069
=====						
Odds ratios:						
	Odds Ratio	...	p-value			
Intercept	9.012	...	0.000			
C(comp_band_code, Treatment(reference='Lead'))[...]	2.817	...	0.003			
C(comp_band_code, Treatment(reference='Lead'))[...]	1.906	...	0.068			
C(comp_band_code, Treatment(reference='Lead'))[...]	1.163	...	0.690			
tenure_years	0.268	...	0.000			
performance_rating	0.629	...	0.000			
overtime_hours_monthly	1.057	...	0.003			
engagement_score	0.843	...	0.001			

[8 rows x 4 columns]

Step 6 — Flight-risk scoring for active employees

The fitted model is used to score every currently active employee's predicted exit probability, then bucket them into Low / Medium / High risk tiers for targeted retention action.

In [6]:

```
import pandas as pd
import numpy as np
import statsmodels.formula.api as smf
import matplotlib.pyplot as plt

df = pd.read_pickle("/home/claude/attrition/clean.pkl")
CH = "/home/claude/attrition/charts"

# Step 6 -- Score ACTIVE employees with the fitted model to flag flight risk

model_df = df.copy()
model_df["comp_band_code"] = pd.Categorical(
    model_df["compensation_band"], categories=["Entry", "Mid", "Senior", "Lead"], ordered=True
)
formula = ("is_exited ~ tenure_years + C(comp_band_code, Treatment(reference='Lead')) + "
           "performance_rating + overtime_hours_monthly + engagement_score")
logit_model = smf.logit(formula, data=model_df).fit(dis=0)

active = model_df[model_df["exit_status"] == "Active"].copy()
active["exit_risk_score"] = logit_model.predict(active)

bins = [0, 0.15, 0.35, 1.01]
labels = ["Low Risk", "Medium Risk", "High Risk"]
active["risk_segment"] = pd.cut(active["exit_risk_score"], bins=bins, labels=labels)

seg_counts = active["risk_segment"].value_counts().reindex(labels)
print("Active employees:", len(active))
print(seg_counts)

colors = {"Low Risk": "#3FBFA6", "Medium Risk": "#1C7293", "High Risk": "#D9704F"}
fig, ax = plt.subplots(figsize=(7, 7))
ax.pie(seg_counts.values, labels=seg_counts.index, autopct="%1.0f%%",
       colors=[colors[s] for s in seg_counts.index],
       wedgeprops={"width": 0.4, "edgecolor": "white"})
ax.set_title("Active Employee Flight-Risk Segments", fontsize=15, fontweight="bold")
plt.tight_layout()
plt.savefig(f"{CH}/risk_segments.png", dpi=150)
plt.close()

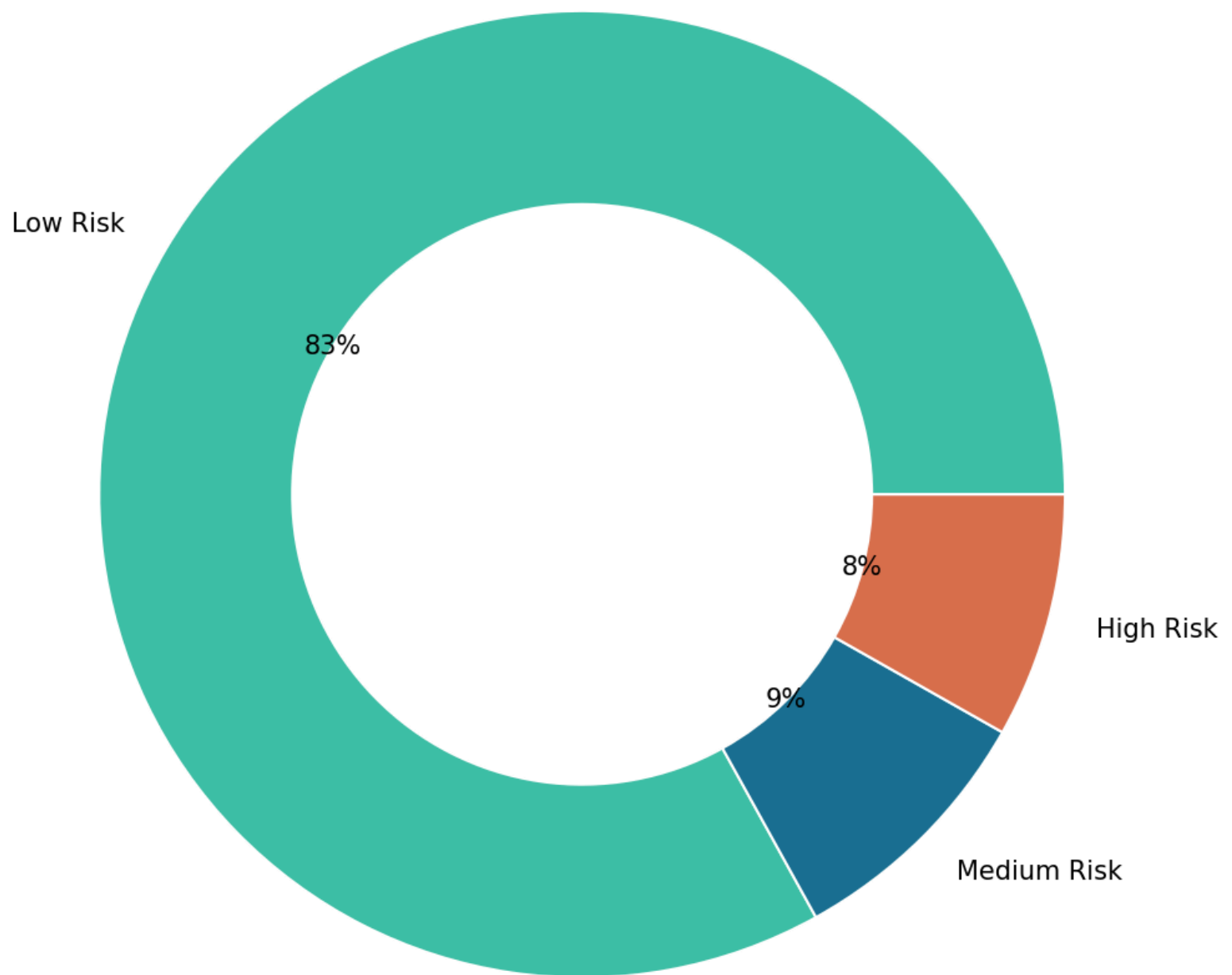
# High-risk department concentration
high_risk_by_dept = (active[active["risk_segment"] == "High Risk"]
                    .groupby("department").size().sort_values(ascending=False))
print("\nHigh-risk headcount by department:\n", high_risk_by_dept)

active.to_pickle("/home/claude/attrition/active_scored.pkl")
seg_counts.to_pickle("/home/claude/attrition/seg_counts.pkl")
```

```
Active employees: 1746
risk_segment
Low Risk      1450
Medium Risk    153
High Risk      143
Name: count, dtype: int64
```

```
High-risk headcount by department:
department
Finance      32
Customer Support  24
Operations    21
Sales         18
HR            17
Marketing     16
Engineering   15
dtype: int64
```

Active Employee Flight-Risk Segments



Recommendations

- 1. Fix the first-year experience** — early-tenure (<1yr) attrition is 34%; strengthen onboarding, mentorship, and 30/60/90-day check-ins.
- 2. Re-examine entry-band pay & growth path** — entry-band employees are 2.8x more likely to exit than lead-band, even controlling for tenure and performance.
- 3. Run stay interviews for the 143 high-risk actives** — concentrated in Finance and Customer Support.
- 4. Cap chronic overtime** — each extra monthly overtime hour raises exit odds ~5.7%.
- 5. Protect engagement, not just pay** — engagement score is independently protective (OR 0.84).